Collaboration and Competition – Tennis Environment

Multi-agent Deep Deterministic Policy Gradient (MADDPG)

1 Introduction

This project solves the Unity Tennis multi-agent reinforcement learning environment using the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm.

In this environment, two agents control rackets and must cooperate to keep a ball in play by hitting it back and forth over a net. The environment is considered solved when the average score over 100 consecutive episodes (taking the maximum score of the two agents per episode) reaches at least +0.5.

The environment was solved in 1143 episodes, achieving an average score of 0.52 over 100 consecutive episodes.

90% of this report has been written with ChatGPT, based on input and python code provided by me.

2 Environment description

The Unity Tennis environment consists of:

- **Number of Agents:** The environment contains two agents
- **State space:** The state space has 24 continuous dimensions. These include position and velocity of the ball and racket.
- **Action space:** The action space has 2 continuous dimensions. Each action corresponds Horizontal or Jump movements. All actions are bounded within [-1, 1].
- **Reward structure:** The agent receives a reward of +0.1 if it hits the ball over the net and -0.01 if the ball hits the ground or goes out of bounds.
- **Solving criterion:** The environment is considered solved when the average score (maximum of scores of both agents) over 100 consecutive episodes is at least +0.5.

3 Learning Algorithm

3.1 Overview of MADDPG

To solve this environment, I implemented the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm.

MADDPG extends the DDPG algorithm to multi-agent settings by using:

- Decentralized actors (each agent has its own policy network)
- Centralized critics (each critic receives the joint state and joint actions of all agents)

This allows each agent to:

- Act using only its own local observations
- Learn using full environment information during training

Each agent has:

- One Actor network
- One Critic network
- Corresponding target networks for both

Experience replay is shared across agents, and soft target updates ($\tau = 0.01$) are used to stabilize learning. Furthermore, states are normalized (per agent) to further stabilize learning.

3.2 Network architectures

3.2.1 Actor network

- Input: 24 dimensions
- Hidden layers:
    - Fully connected layer with 400 units (ReLU)
    - Fully connected layer with 300 units (ReLU)
    - Output layer: 2 units with tanh activation. The tanh output ensures actions remain in the valid range [-1, 1].
- Weights were initialized following the original DDPG paper:
    - Hidden layers initialized uniformly in [-1/√f, 1/√f]
    - Final layer initialized in [-3e-3, 3e-3]

3.2.2 Critic Network

- Input:

    - Joint State size (24 * 2 = 48 dimensions)
    - Joint Action size (2 * 2 = 4 dimensions)
- Architecture:
    - First fully connected layer processes state only (400 units, ReLU)
    - Second fully connected layer takes concatenated state features and joint actions (300 units, ReLU)
    - Output layer produces a single Q-value (Q(s,a))

The critic estimates the Q-value of the joint state-action pair

3.3 Hyperparameters

The following hyperparameters were used:

- **BUFFER_SIZE = 1e5**
  Size of the replay buffer. Stores past experiences for off-policy learning.
- **BATCH_SIZE = 128**
  Number of samples drawn from replay buffer per learning step.
- **GAMMA = 1.0**
  Discount factor. Since episodes are relatively short and cooperative behavior is required, no discounting was applied.
- **TAU = 1e-2**
  Soft update interpolation parameter for target networks. A relatively larger value was used for faster target tracking.
- **LRA = 1e-4**
  Learning rate for the Actor network.
- **LRC = 3e-3**
  Learning rate for the Critic network. Slightly higher than the actor learning rate to stabilize Q-learning.
- **UPDATE_EVERY = 1**
  Learning is performed at every timestep.
- **BUFFER_WARMUP = 10000**
  Number of experiences collected before learning begins.
- **LEARN_STEPS = 2**
  Number of gradient updates per environment step.
- **HIDDEN_ACTOR = [400, 300]**
  Actor network layer sizes.
- **HIDDEN_CRITIC = [400, 300]**
  Critic network layer sizes.
- **NOISE_SIGMA = 0.3**
  Standard deviation of Gaussian exploration noise.
- **INFLATE_REWARDS = 100**
  Rewards were multiplied by 100 during training.

3.4 Exploration strategy

The original DDPG paper proposes using an Ornstein–Uhlenbeck (OU) process for exploration noise to generate temporally correlated noise suitable for physical control problems. In this implementation, I instead used Gaussian noise:

$$\mu'(s_t) = \mu\left(s_t \middle| \theta_t^{\mu}\right) + \mathcal{N}(0, \sigma^2) \times f_t$$

with $\sigma = 0.3$ scaled by a decaying noise factor f_t. The noise scaling factor starts at 1 and decays with a factor 0.999 every episode. $\mu'(s_t)$ is clipped between $[-1; 1]$

Reasoning:

- Gaussian noise is simpler to implement.
- Empirically, it performs similarly to OU noise in many modern continuous control tasks.
- The Tennis environment does not strongly require temporally correlated noise.
- Recent literature often reports negligible difference between OU and Gaussian noise in practice.

In this project, Gaussian noise produced successful convergence. Nevertheless, it would be worthwhile to investigate the performance when OU noise would have been added.

3.5 Reward scaling

The environment rewards are very small:

- +0.1 for hitting the ball over the net
- -0.01 if the ball hits the ground or goes out of bounds

During early debugging, I observed extremely small gradients (on the order of 1e-6 to 1e-7), which resulted in nearly zero learning progress.

To address this, rewards were multiplied by a factor of 100 during training. This:

- Increased gradient magnitudes
- Improved critic learning stability
- Prevented vanishing update steps
- Did not change the optimal policy (only rescales Q-values)

This modification significantly improved learning behavior.

3.6 State Normalization

A key improvement was the introduction of running state normalization. A running estimate of mean and variance across all observed states was maintained for each agent separately. States were normalized before being fed into the actor and critic networks:

$$s_{normalized} = \frac{s - \mu}{\sigma + \epsilon}$$

This improved training stability and learning speed by ensuring well-conditioned network inputs.

State normalization is not performed independently for each individual state sample, nor is it computed from a single minibatch. Instead, a running estimate of the mean and variance is maintained across the entire replay buffer, reflecting the global distribution of observed states per agent. As new experiences are added, the running statistics are updated and all sampled states are normalized using the updated global mean and variance, ensuring consistency and stability throughout training.

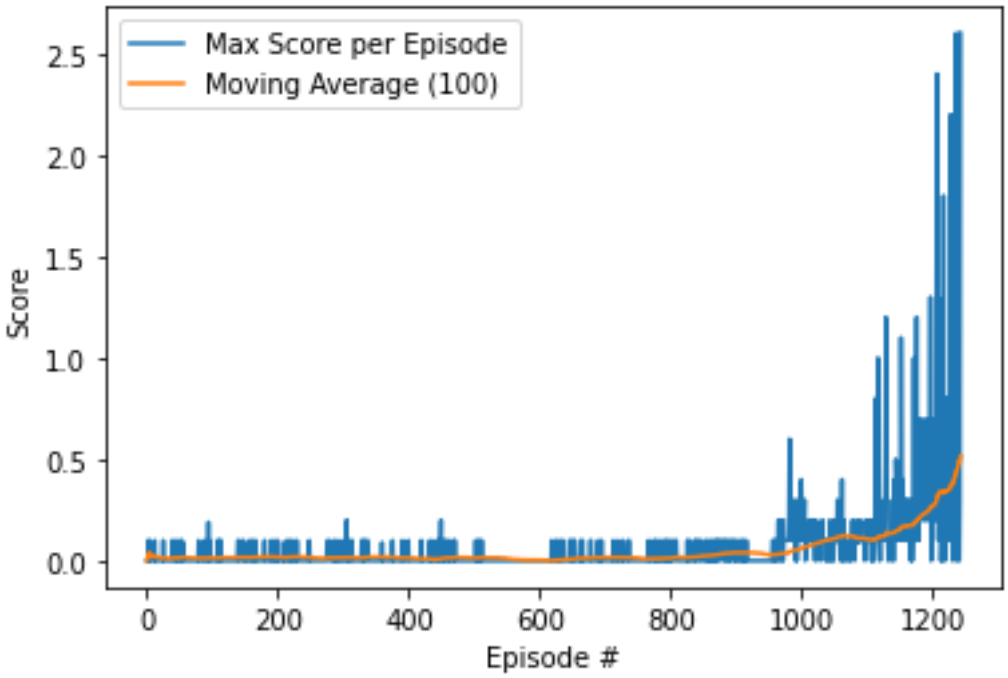
4 Results

The agent successfully solved the environment.

- Number of episodes required: 1143
- Final 100-episode average score: 0.52

The training appears to be somewhat unstable due to taking the maximum over 2 agents. Nevertheless, from 1000 episodes onward the learning curve was very steep, going from an average score of 0.11 to 0.52 in 143 episodes. To reflect this, a moving average of the scores is shown in the graph below as well (orange line).

```
Episode 100        Average Score: 0.01
Episode 200        Average Score: 0.02
Episode 300        Average Score: 0.01
Episode 400        Average Score: 0.02
Episode 500        Average Score: 0.01
Episode 600        Average Score: 0.00
Episode 700        Average Score: 0.02
Episode 800        Average Score: 0.01
Episode 900        Average Score: 0.04
Episode 1000       Average Score: 0.06
Episode 1100       Average Score: 0.11
Episode 1200       Average Score: 0.27
Episode 1243       Average Score: 0.52
Environment solved in 1143 episodes!     Average Score: 0.52
```



5 Ideas for Future Work

Several improvements could potentially enhance performance and stability:

1. Parameter noise instead of action noise
   Could improve exploration in continuous control tasks.

2. Prioritized Experience Replay
   Sampling more informative experiences may speed up convergence.

3. Shared actor networks
   Sharing parameters between agents may improve sample efficiency due to symmetry. Note that I started implementing this, but did not finish it.

4. Twin critics (TD3-style stabilization)
   Using two critics and taking the minimum target value could reduce Q-value overestimation.

5. Curriculum learning
   Gradually increasing task difficulty could improve early training stability.

6. More systematic reward scaling
   Instead of fixed inflation, adaptive normalization could be applied.

6 Conclusion

This project demonstrates successful application of MADDPG to a cooperative multi-agent continuous control problem.

Key implementation choices included:

- Centralized critics with decentralized actors
- Gaussian exploration noise instead of OU noise
- Reward scaling to address vanishing gradients
- State normalization per agent
- Multiple learning steps per environment interaction

The final agent achieved stable cooperative behavior and met the environment's solving criterion.